

Apprentissage automatique avancé : Deep Learning

Stéphane Ayache

Polytech Marseille



QARMA



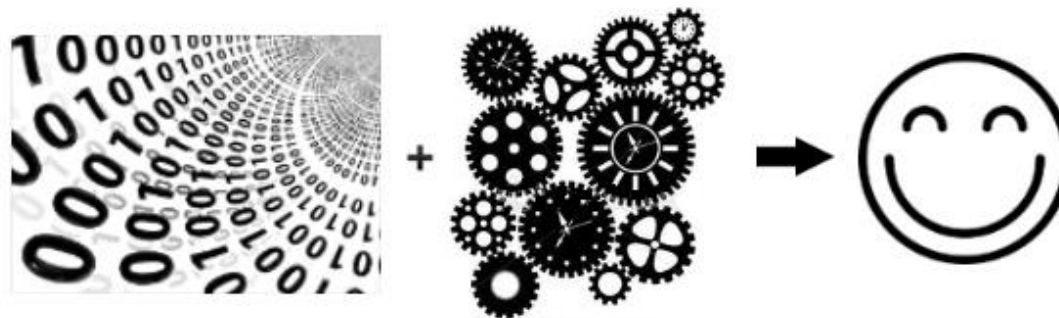
Apprentissage automatique (Machine Learning)

⇒ Apprendre à résoudre une tâche qu'un algorithme classique ne permet pas

- Supervisé : à partir d'exemples (données étiquetées)



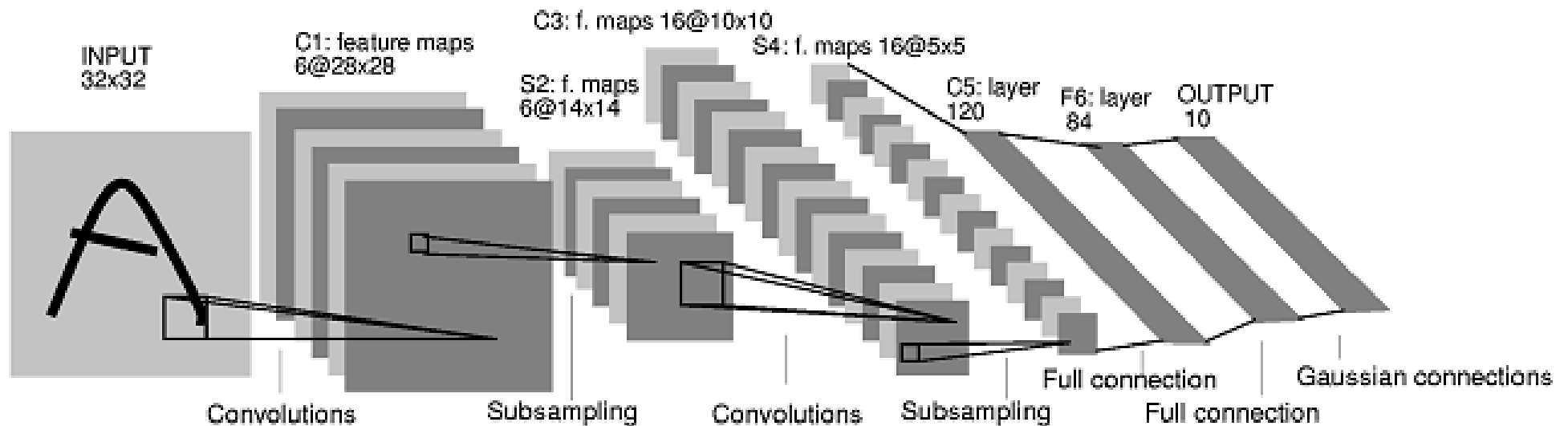
- "Big data" : beaucoup de données disponibles → nombreuses applications en apprentissage automatique avec des performances inégalées !



Deep learning

- Modèle d'apprentissage de représentations en couches, historique depuis les années 1950s
- Des variantes pour de nombreux paradigmes d'apprentissage : supervisé, non-supervisé, classification, régression, renforcement, ...
- Nécessite beaucoup de données d'apprentissage, nombreux paramètres à optimiser
 - Adapté au « Big Data » : optimisation par descente de gradient stochastique
- Plusieurs améliorations dans les années 1990s, nombreuses "révolutions" depuis 2010 : vision, parole, texte, traduction, ...

Réseau de neurones (profond)



Bref historique

1957 (Rosenblatt) Perceptron

1969 (Minsky, Papert) Problème XOR

1986 (Rumelhart et al) MLP et Backpropagation

1992 (Vapnik et al) SVM

1998 (LeCun et al) ConvNets

2010 (Hinton et al) Deep Neural Networks

2012 (Krizhevsky, Hinton et al) AlexNet - 8 couches

2014 GoogleNet - 22 couches

2016 ResidualNet – 152 couches

2017 AlphaGo !

Des succès spectaculaires en Vision, NLP, IA

Données structurées spatial et/ou temporel

Quantités importantes

De nombreux acteurs privés s'y sont intéressés depuis mi 2010

Google, Meta, Amazon, Apple

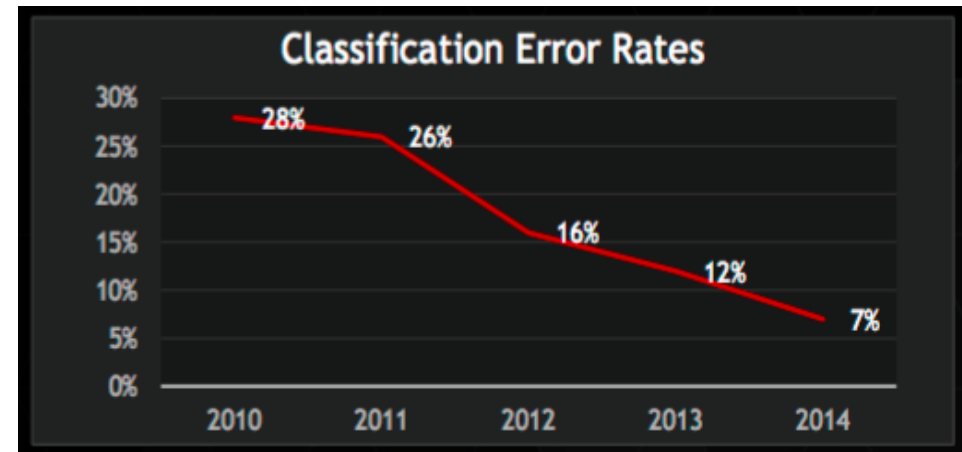
« IA générative » depuis 2017-2020

Language models, Adversarial NN, Diffusion models, Transformers, ...

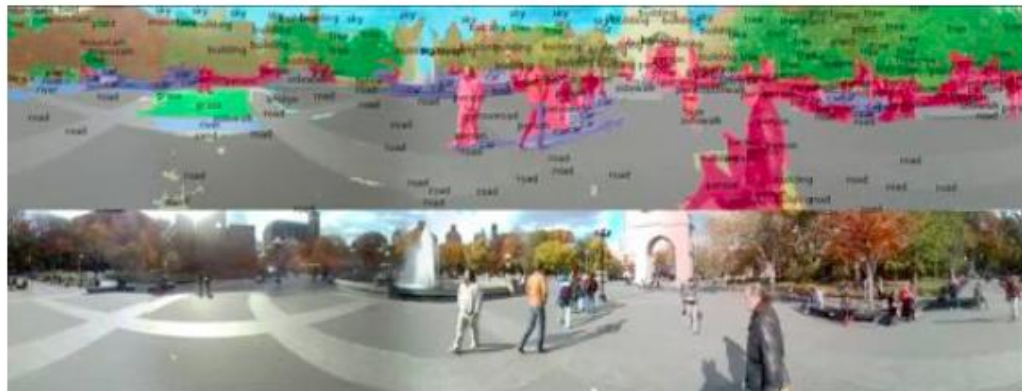
Aujourd'hui IA = chatbot VLM

Deep learning – les débuts en vision

- Classification d'images
⇒ distinguer 1000 classes d'images parmi un million (ImageNet Challenge)

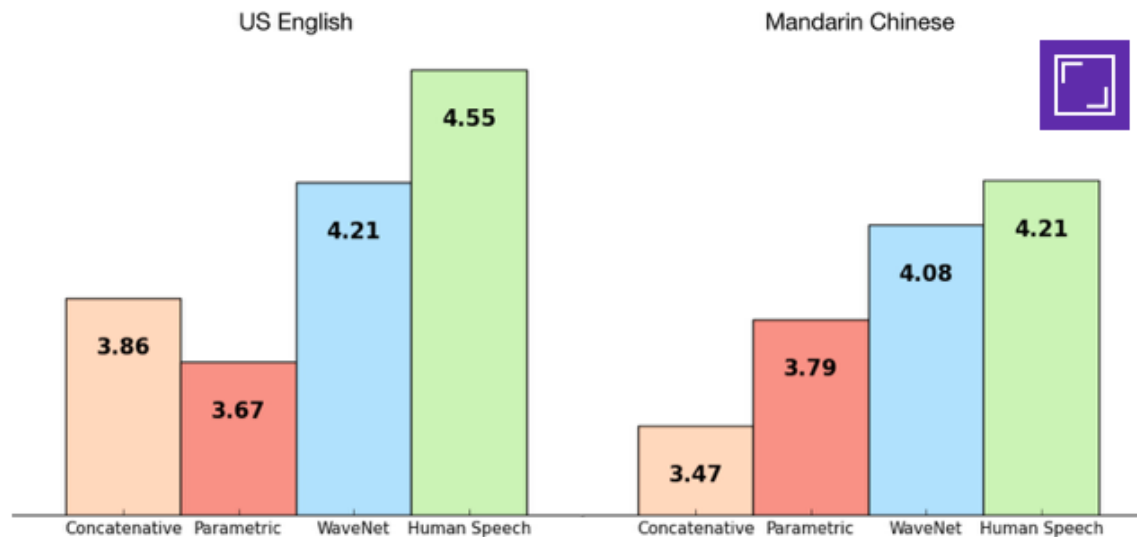


- détection d'objets, analyse de scènes, analyse vidéo, segmentation, génération d'images, ...



Deep learning - en synthèse de la parole

- « Wavenet: A generative model for raw audio » [DeepMind]
- <https://deepmind.com/blog/wavenet-generative-model-raw-audio/>



Deep Learning - en IA

- « Mastering the game of Go with deep neural networks and tree search » [DeepMind]
- AlphaGo : apprentissage par renforcement + CNN

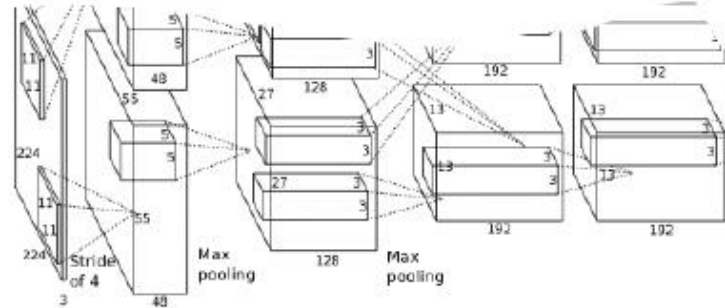


Deep Learning - Réseau de neurones profonds



Big Data: ImageNet

+



Deep Convolutional Neural Network

+

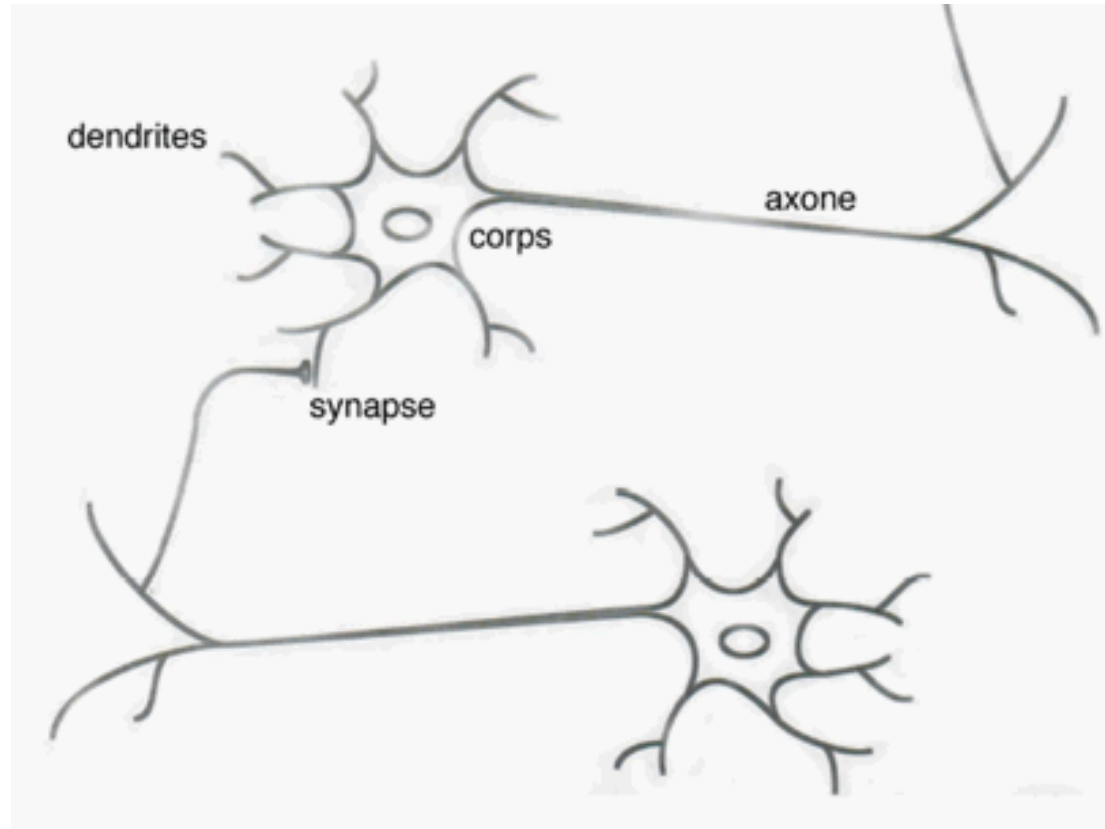


Backprop on GPU



Learned Weights

Neurone, Perceptron, ...



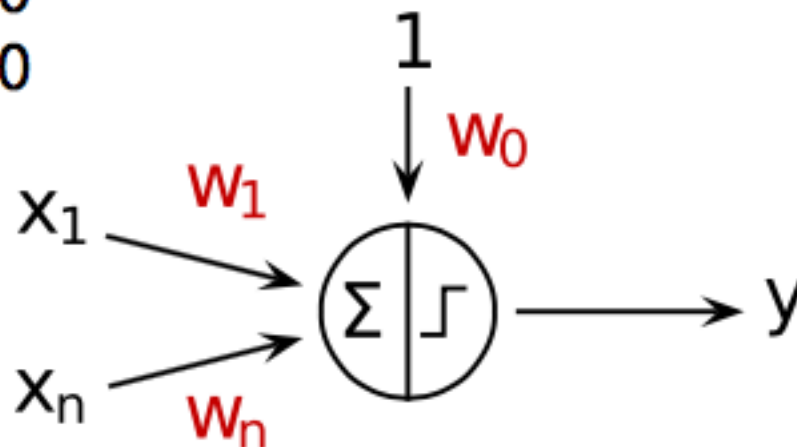
Inspiration biologique : les neurotransmetteurs captés par les dendrites s'accumulent, puis un message est libéré vers l'axone lorsqu'un potentiel d'action est atteint.

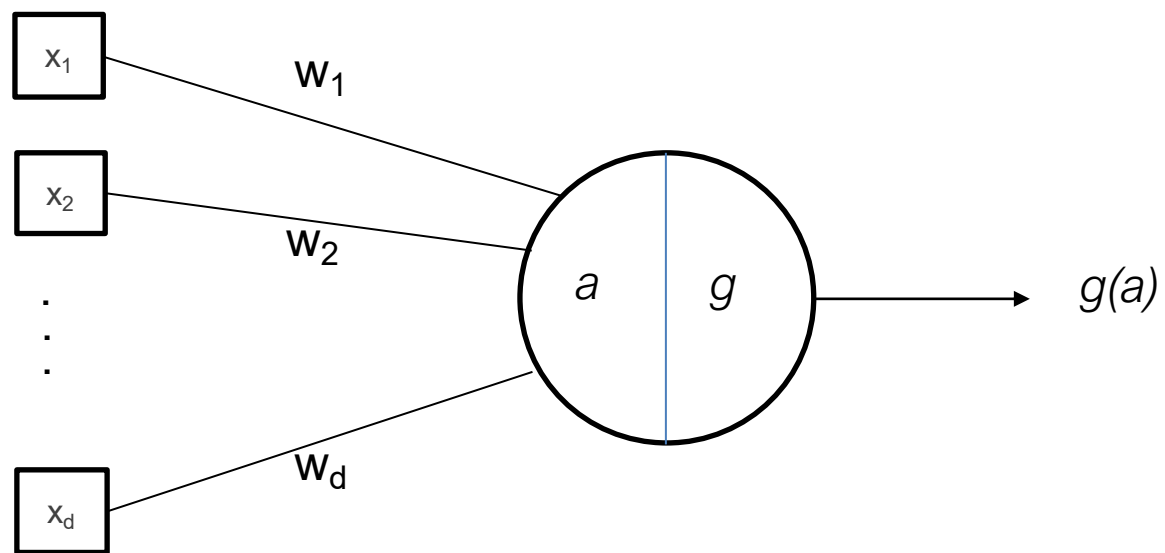
Neurone, Perceptron, ...

→ Neurone $\approx$ Perceptron = fonction linéaire

$$(x_1, \dots, x_n) \mapsto y = f\left(w_0 + \sum_{i=1}^n (w_i x_i)\right) = f\left(w_0 + \langle \vec{w}, \vec{x} \rangle\right) \text{ avec}$$

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$





Neurone, Perceptron, ...

→ Algorithme d'apprentissage d'un Perceptron

Algorithme d'apprentissage du Perceptron

Entrée : $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_l, y_l)\}$, un échantillon complété linéairement séparable de $\mathbb{R}^{n+1} \times \{-1, 1\}$

$\mathbf{w}_0 = \mathbf{0} \in \mathbb{R}^{n+1}, k = 0$

Répéter

Pour $i = 1$ à l

Si $y_i \langle \mathbf{w}_k, \mathbf{x}_i \rangle \leq 0$ **alors**

$\mathbf{w}_{k+1} = \mathbf{w}_k + y_i \mathbf{x}_i$

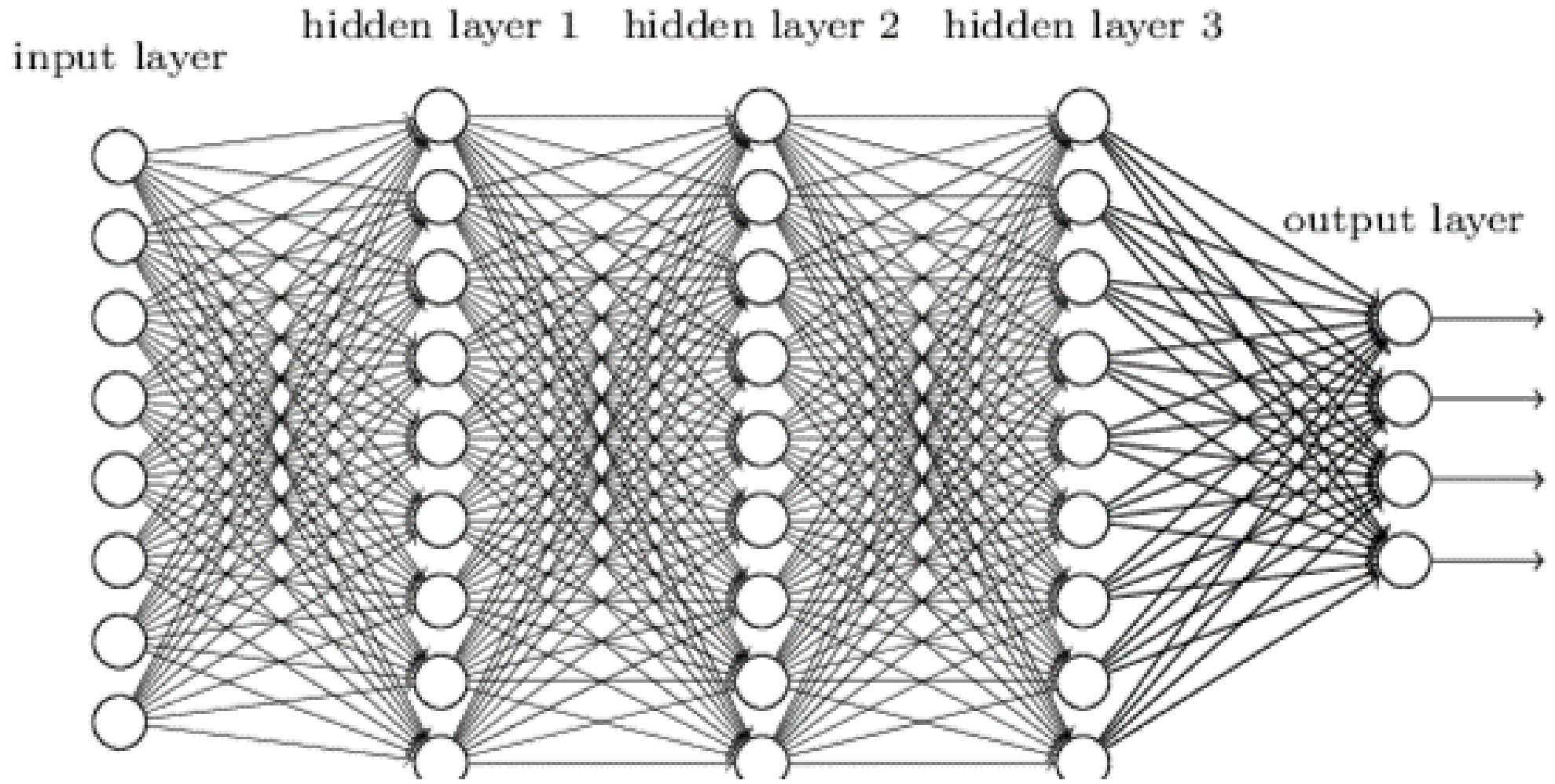
$k = k + 1$

FinPour

Jusqu'à ce qu'il n'y ait plus d'erreurs

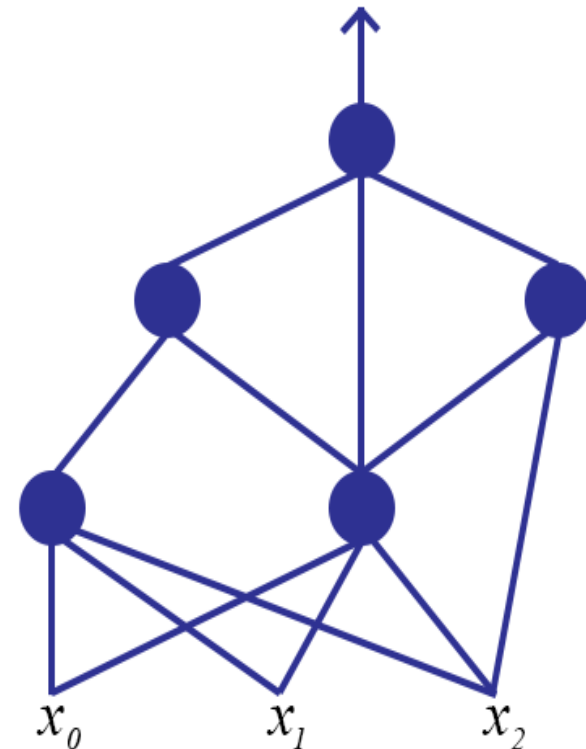
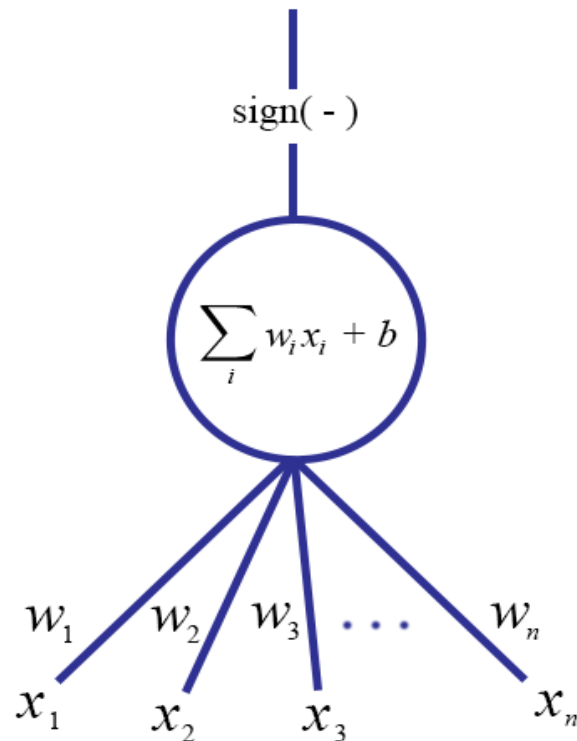
Sortie : $\mathbf{w}_k$

Deep Learning - Réseau de neurones profonds



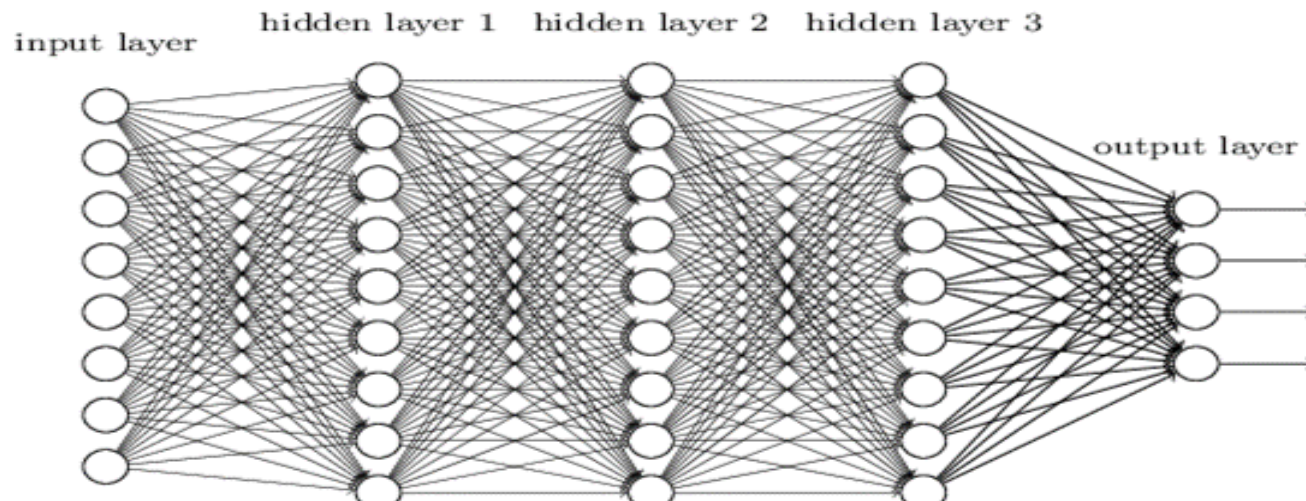
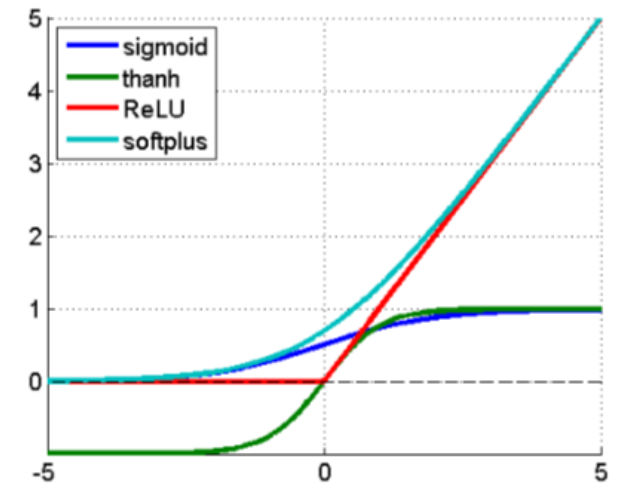
MLP

- Un seul neurone $\rightarrow$ Perceptron $f = g(\langle w^T, x \rangle)$
- Une seule couche de neurones $\rightarrow$ Perceptrons $f = g(W \cdot x)$
- Plusieurs couches $\rightarrow$ Multi Layer Perceptron $f = g(W \cdot g(W_h \cdot x))$



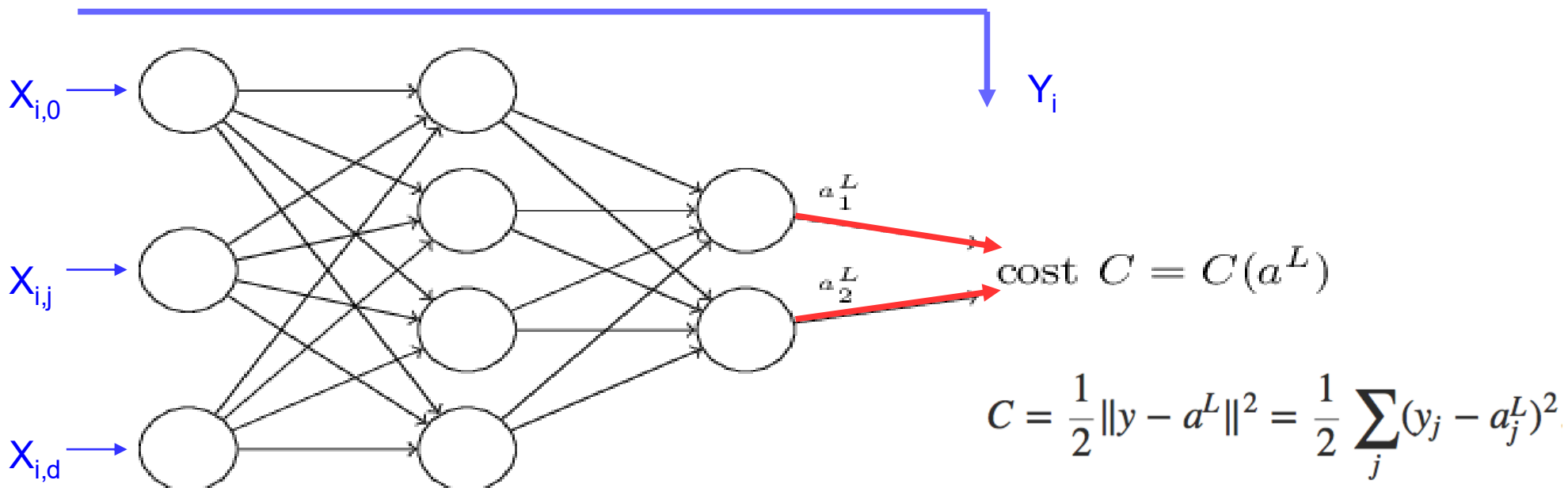
MLP

- Non linéarités dérivables
 - sigmoid, tanh
- ReLU : $\max(0, x)$ [Glorot, 2011]
- Structure en couches



Architecture d'un MLP

- Selon la nature du problème : classification vs régression
 - Choix d'une fonction de coût (loss function)
 - Choix d'une fonction d'activation sur la couche de sortie
- Couche d'entrée de taille = dimension des données
- Couche de sortie selon le problème
- Nombre et taille des couches cachées à déterminer
 - expertise, validation croisée, architectures connues, ...



Quelques fonctions de coûts

MSE : Mean Squared Error (= L2 loss)

Regression, Classification

MAE : Mean Average Error (= L1 Loss)

$$\sum_j (y_j - a_j^L)^2$$

Binary Cross Entropy

Classification binaire

$$-(y \log(p) + (1 - y) \log(1 - p))$$

Categorical Cross Entropy

Multiclasse

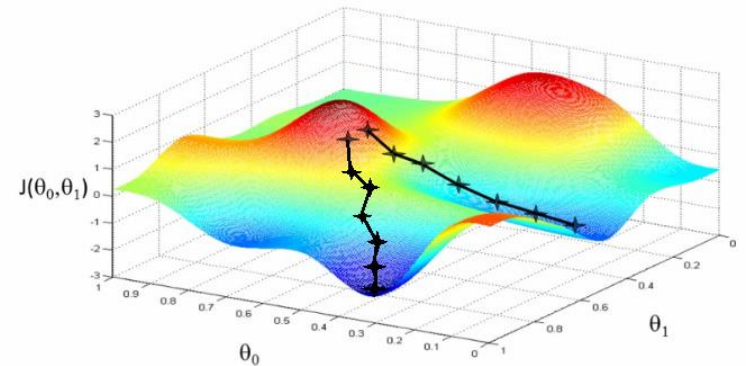
$$-\sum_{c=1}^M y_{o,c} \log(p_{o,c})$$

➔ Attention à la fonction d'activations sur la dernière couche !

- Softmax pour Cross Entropy

Entraînement d'un MLP

- Minimisation de la fonction de coût
 - $\arg \min_{\theta} L(f_{\theta}, S) = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x^i), y^i)$
 - Somme de fonctions dérivables
 - Non convexe
 - L'erreur dépend des poids
- Stochastic Gradient Descent (SGD)
 - Plusieurs variantes améliorées de SGD
 - Apprentissage par minibatches
- Back-propagation
 - Rétropropagation de l'erreur pour mettre à jour les paramètres des couches cachées
 - Problème du « vanishing gradient »



Descente de gradient

Minimisation d'une somme de fonctions différentiables

Minimise l'erreur en fonction des paramètres θ

Suit la direction opposée au gradient

cf fonctions tangentes et dérivées partielles

Peu converger vers un minimum local

Algorithme itératif

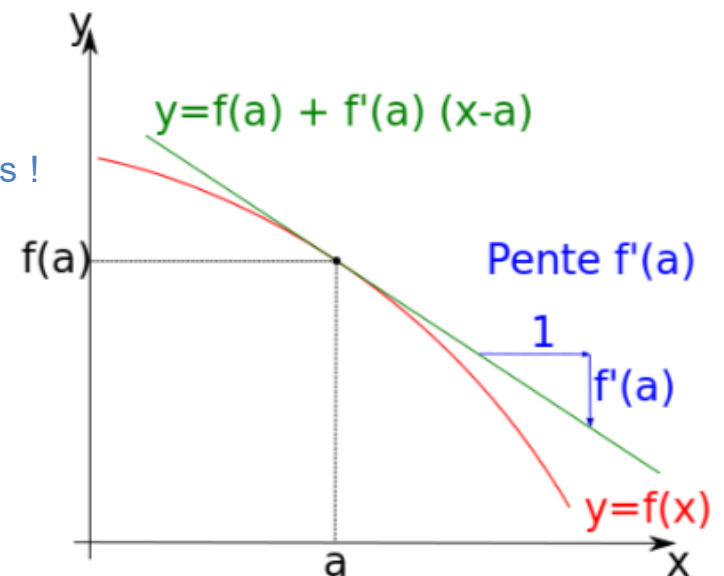
$\theta^0 \leftarrow$ initialisé aléatoirement

Répéter

$$\theta^{t+1} \leftarrow \theta^t - \alpha \frac{\partial L(f_\theta, S)}{\partial \theta}$$

Tant que $\left\| \frac{\partial L(f_\theta, S)}{\partial \theta} \right\| > \epsilon$ (n'est pas proche de 0)

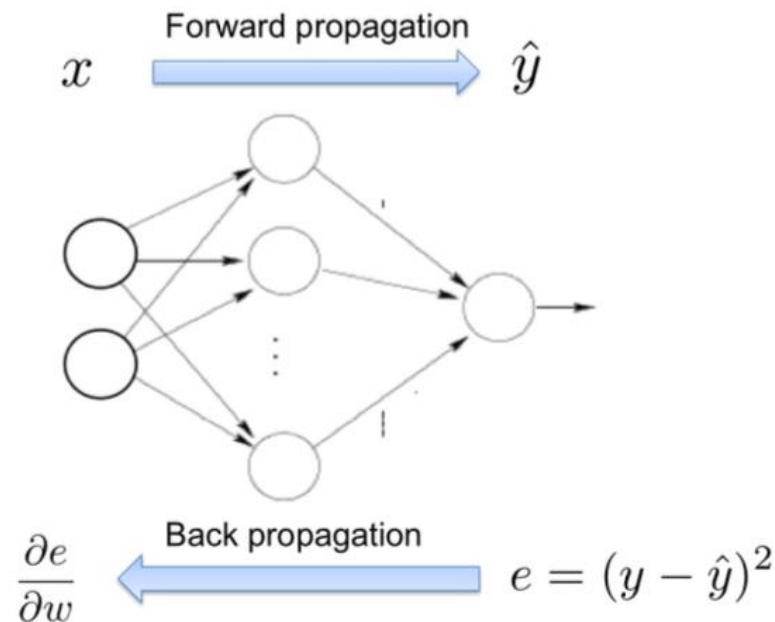
$S =$ tous les exemples !



Back Propagation

- Calculer le gradient par rapport aux paramètres des couches intermédiaires
 - Utilisation de dérivées partielles
 - Chain Rule

$$(g \circ f)' = (g' \circ f) \times f' \quad \rightarrow \quad \frac{d(g \circ f)}{dx} = \frac{dg}{df} \frac{df}{dx}$$



Back Propagation

- Evaluation de l'erreur de la couche de sortie

- MSE : $C = \frac{1}{2} \|y - a^L\|^2 = \frac{1}{2} \sum_j (y_j - a_j^L)^2$

- Dérivée : $\frac{\partial C_k(W)}{\partial y_i} = 2(y_i - d_i)$

- % fct d'activation : $\delta_i = \frac{\partial C_k(W)}{\partial a_i} = \frac{\partial C_k(W)}{\partial y_i} \frac{\partial y_i}{\partial a_i} = 2(y_i - d_i) g'(a_i)$

- % poids w_{ij} de la $j^{\text{ème}}$ cellule de la couche cachée L-1 vers la $i^{\text{ème}}$ cellule de sortie L :

$$\frac{\partial C_k(W)}{\partial w_{ij}} = \frac{\partial C_k(W)}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} = \delta_i \times y_j$$

Back Propagation

- Evaluation de l'erreur d'une couche intermédiaire

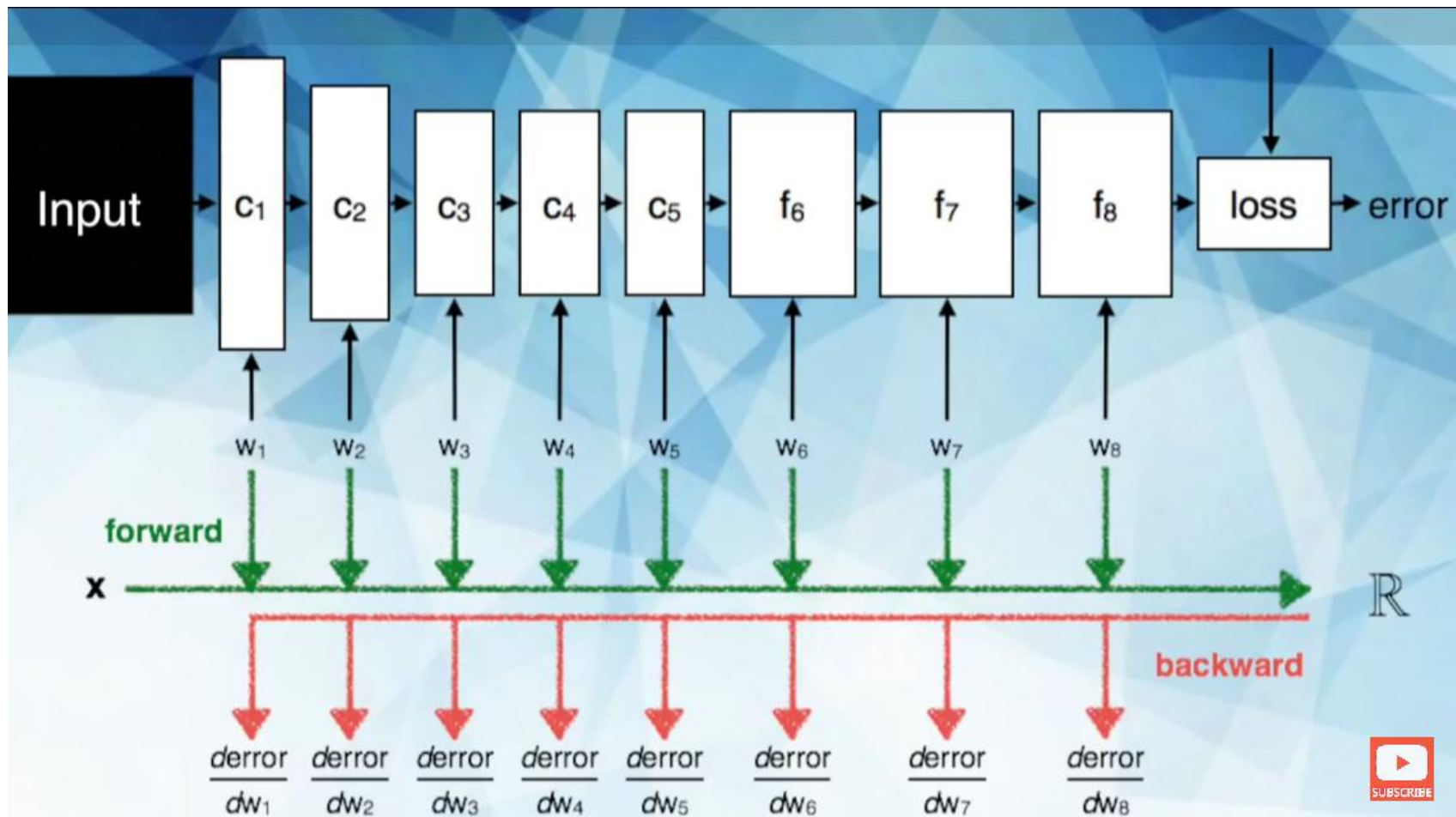
- $\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$

- % poids : $\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$

- Mise à jour des poids

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

Back Propagation - illustration



Descente de gradient

$$\arg \min_{\theta} L(f_{\theta}, S) = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x^i), y^i)$$

Comment apprendre avec les n exemples ?

Batch $\rightarrow$ souvent impossible si trop d'exemples

Minibatch $\rightarrow$ choisi plein de fois aléatoirement $m \ll n$ exemples

Stochastic $\rightarrow$ taille minibatch = 1

Minibatch et Stochastic introduisent de l'aléatoire

Effet de régularisation, meilleure convergence

Ne trouve pas toujours la meilleure solution

Sensible à l'initialisation

Paramètres learning step, momentum, decay, ...

Réseaux de neurones profonds ?

Plusieurs types de couches

- Couche linéaire
 - Fully-connected : $f(x) = Wx + b$
- Non-linéarité
 - tanh, sigmoid, ReLU (Rectified Linear Unit), ...
- Dropout
 - Ajoute du bruit : régularisation, réduit le sur-apprentissage
- Batch Normalisation
 - Normalise les activités $\sim N(0, 1)$
 - Réduit le vanishing gradient
- Convolution
 - Taille des filtres ? Pas entre les filtres (stride) ?
- Pooling
 - Quelle stratégie, quel sous-échantillonnage ?

Couche Dense

« Fully connected layer »

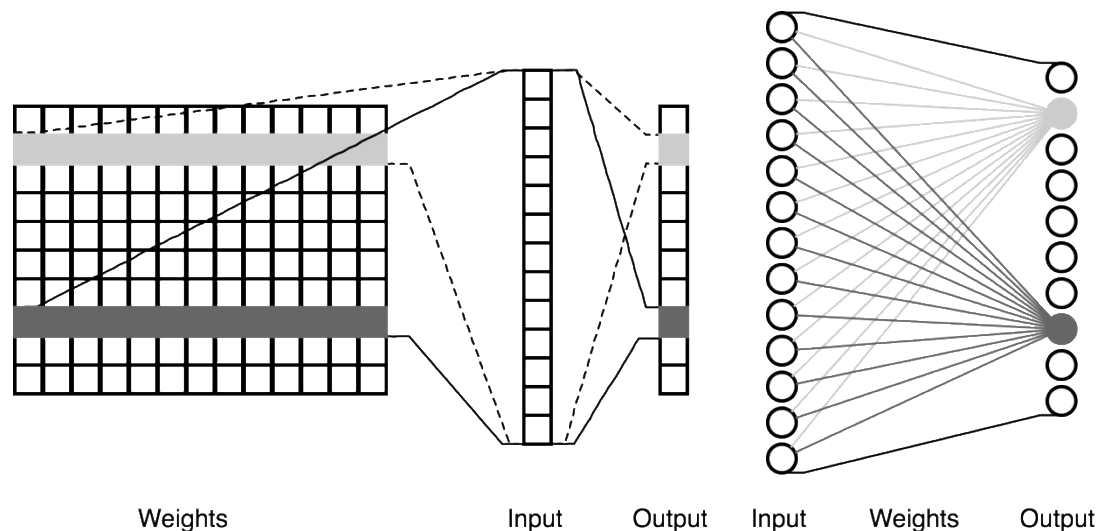
Tous les neurones d'entrée sont connectés aux neurones de sorties

Opération linéaire : $y = W.x + b$

$\text{nb_inputs} * \text{nb_outputs}$ poids (+ nb_outputs) $\rightarrow$ peut être très gros

Souvent utilisé en classifieur linéaire

Utilisation en représentation, avec non-linéarités



Moyennage de modèles

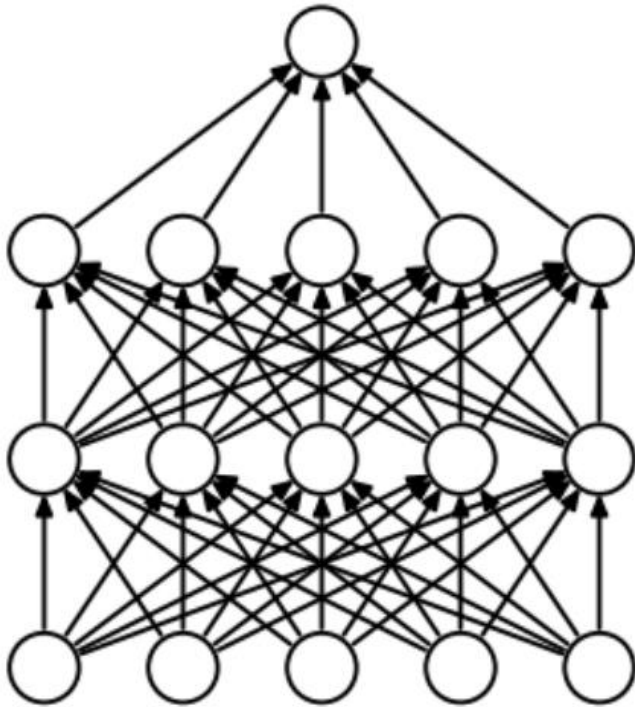
Une technique pour améliorer les performances de classifieur(s)

- Idée
 - Méthode ensembliste : combine (par exemple en faisant voter) plusieurs classifieurs différents appris pour le même problème
- Combinaison
 - Votes éventuellement pondérés, agrégation de scores
- Obtenir des classifieurs diversifiés
 - Plusieurs initialisations
 - Complexité variable (#neurones sur la Couche Cachée)
 - Bagging: Réchantillonnage de la base d'apprentissage avec remise
 - Boosting: Apprentissage séquentiel focalisant sur les exemples mal classés. Poids de combinaison optimaux.

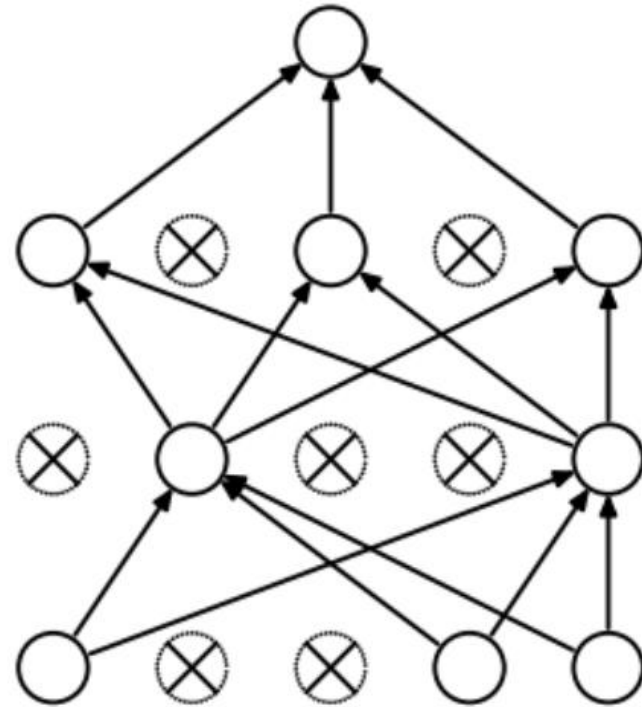
Dropout

- **Lors de l'apprentissage, à chaque itération (gradient stochastique)**
 - Lors de la propagation
 - On inhibe (i.e. on annule la sortie) tous les neurones de la couche d'entrée et des couches cachées avec une certaine probabilité
 - On rétropropage l'erreur normalement
 - **En test**
 - On pondère les poids et on utilise normalement le RN
- **Cela revient à apprendre de nombreux modèles partageant des paramètres et à les moyenner**

Dropout



(a) Standard Neural Net



(b) After applying dropout.

Supervisé vs non-supervisé vs Finetuning

- L'apprentissage non-supervisé des couches permet de les (pré-) apprendre
 - Why Does Unsupervised Pre-training Help Deep Learning?
[D. Erhan, Y. Bengio, A. Courville, P-A. Manzagol, P. Vincent, S. Bengio, 2010]
 - Utilisation d'un auto-encoder
- Finetuning
 - How transferable are features in deep neural networks?
[J. Yosinski, J. Clune, Y. Bengio, H. Lipson, 2014]
 - Utilisation d'un modèle pré-entraîné (sur une autre tâche et/ou dataset)
 - Affiner des couches ; Apprendre de nouvelles couches
 - Permet du transfert-learning

Implémentation

Les toolbox facilitent les calculs de gradients

Numerical differentiation

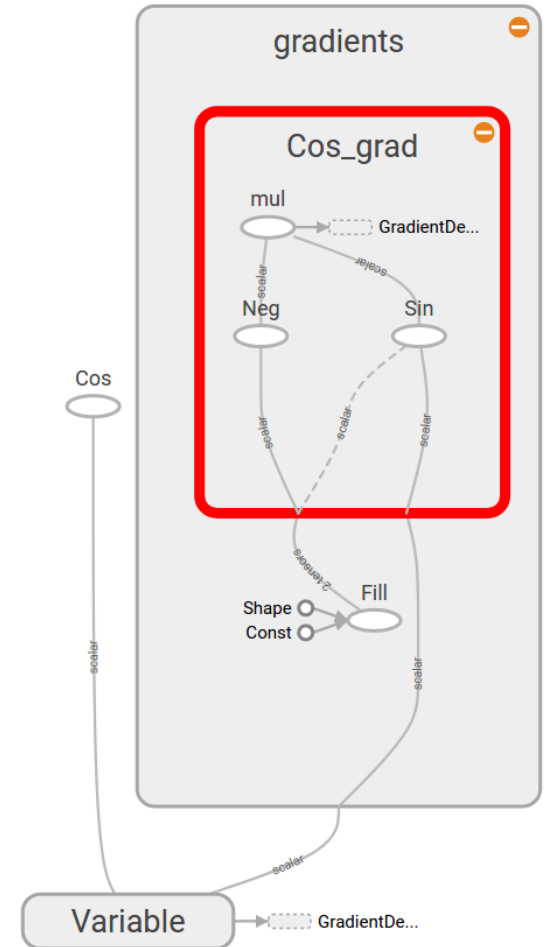
Symbolic differentiation

Automatic differentiation

Structure des opérations en graph

BackProp & Auto diff

<https://www.youtube.com/watch?v=-yhm3WdGFok>



Automatic differentiation

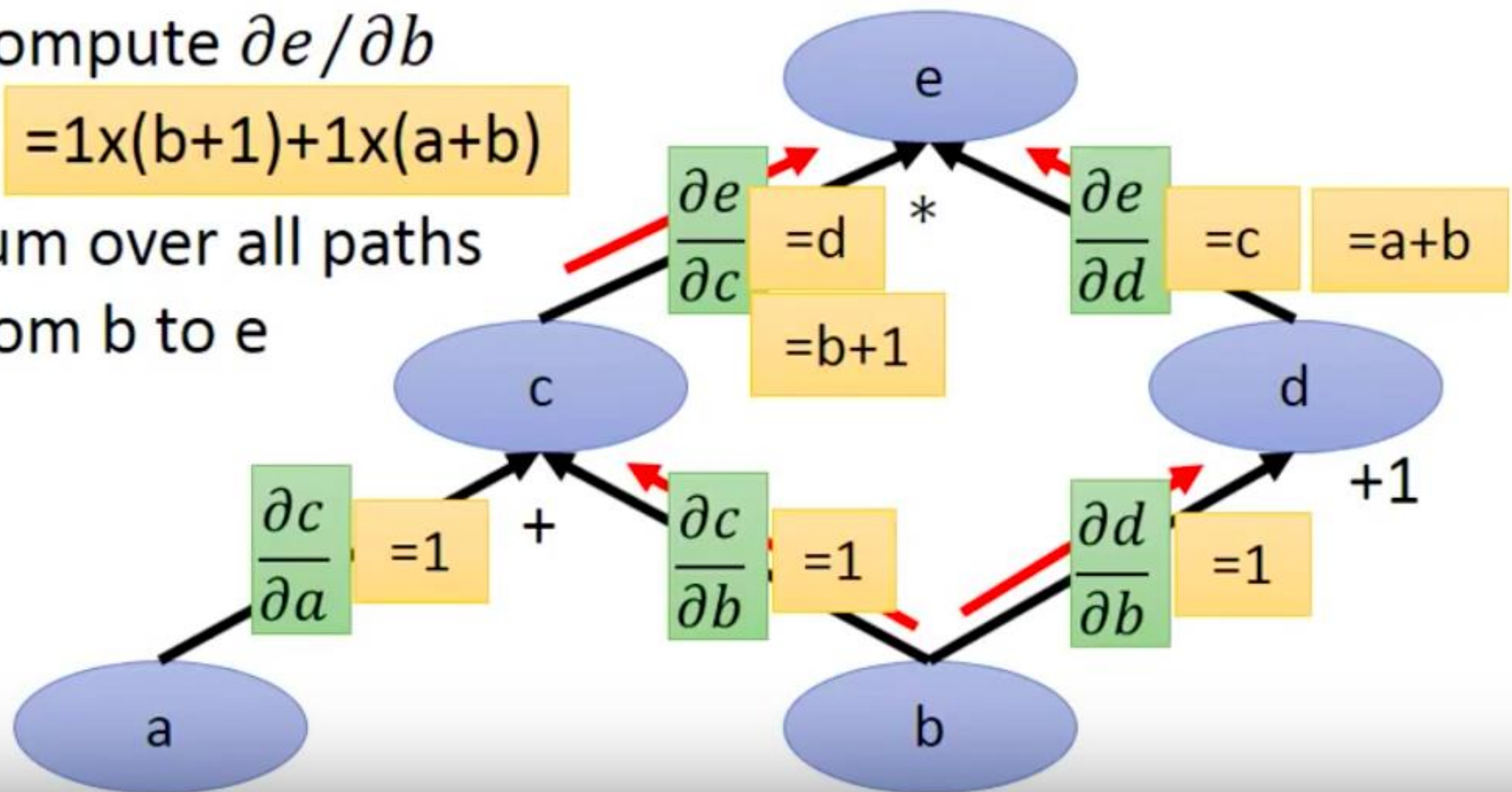
- Example: $e = (a+b) * (b+1)$

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

Compute $\partial e / \partial b$

$$= 1 \times (b+1) + 1 \times (a+b)$$

Sum over all paths
from b to e



Toolboxes

- Les principales librairies python
 - Tensorflow → Google
 - Keras → Google
 - Surcouche de Tensorflow et Theano
 - Pytorch → Meta
- Calculs sur GPU grâce au drivers NVIDIA (CUDA, CUDNN)

Les TPs

- Notebooks
- Ensemble pendant les séances
- A continuer / finir pour les séances suivantes
- Utilisation de Keras / Tensorflow